

Program 4

Title: Integration of Git with Docker for Source-Controlled Application Builds

Introduction

In modern software engineering, version control systems, containerization, and automated deployment play a critical role in application development. Git is widely used to manage and track changes in source code, enabling collaboration and maintaining version history. Docker, on the other hand, allows developers to create lightweight, portable containers that ensure applications run consistently across different environments.

By integrating Git with Docker, application builds become reproducible, traceable, and automated. Each Docker image is built directly from source code stored in a Git repository, ensuring that the exact version of the application can be rebuilt at any time. This integration forms the foundation of CI/CD pipelines, DevOps practices, and cloud-native deployments.

This documentation describes the process of integrating Git with Docker to create source-controlled application builds, illustrated using workflow screenshots.

Project Structure

Program4/

- **index.js** # Main application source file
- **package.json** # Project metadata and dependencies
- **package-lock.json** # Locked dependency versions
- **Dockerfile** # Docker build instructions
- **node_modules/** # Installed Node.js dependencies



```
GNU nano 7.2 Dockerfile
FROM node:18 AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

FROM node:18-slim

WORKDIR /app

COPY --from=build /app /app

EXPOSE 3000

CMD ["node", "index.js"]

Read 19 lines
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

This screenshot displays the contents of the Dockerfile opened in the nano text editor. The Dockerfile defines the instructions Docker follows to build and package the application into a container image.

Multi-Stage Build:

The Dockerfile uses an advanced approach known as a multi-stage build, which helps reduce the final image size and improves performance.

Stage 1 (FROM node:18 AS build):

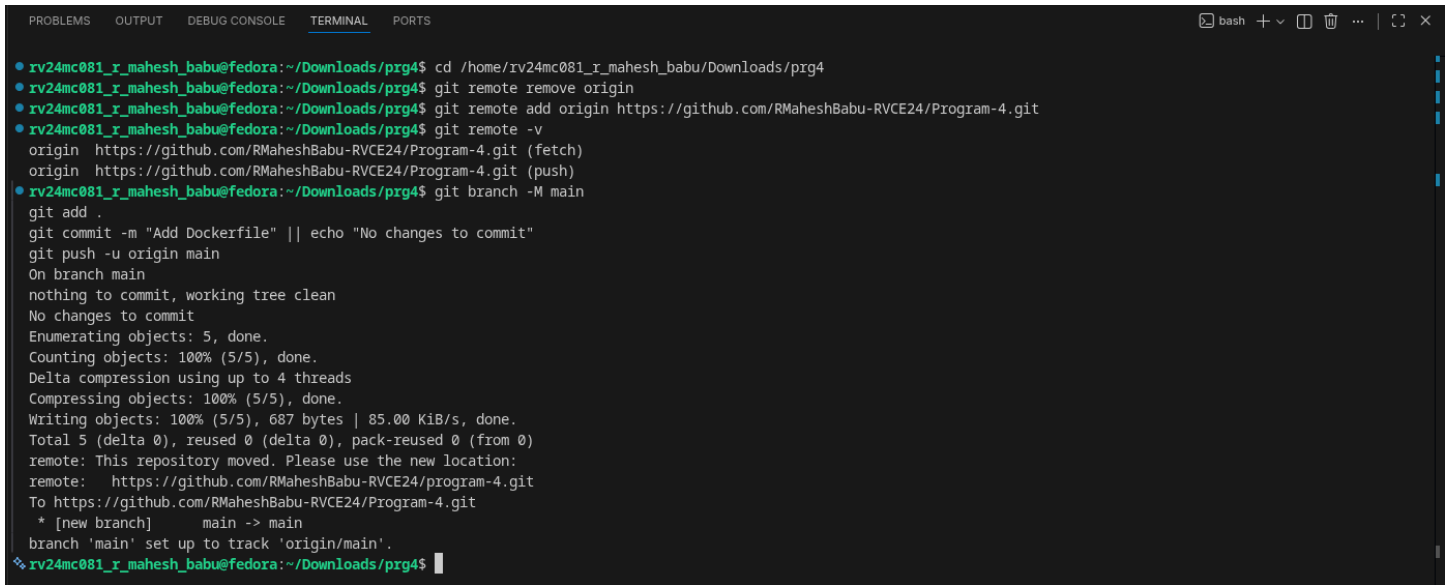
In the first stage, a complete Node.js base image is used. The application source files are copied into the container, and npm install is executed to download and install all required dependencies.

Stage 2 (FROM node:18-slim):

In the second stage, a lightweight slim version of the Node.js image is selected. Only the necessary application files and dependencies are copied from the build stage using the COPY --from=build instruction.

Configuration:

The Dockerfile exposes port 3000 to allow external access to the application and specifies the startup command to run the application using node index.js.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$ cd /home/rv24mc081_r_mahesh_babu/Downloads/prg4
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$ git remote remove origin
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$ git remote add origin https://github.com/RMaheshBabu-RVCE24/Program-4.git
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$ git remote -v
origin https://github.com/RMaheshBabu-RVCE24/Program-4.git (fetch)
origin https://github.com/RMaheshBabu-RVCE24/Program-4.git (push)
• rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$ git branch -M main
git add .
git commit -m "Add Dockerfile" || echo "No changes to commit"
git push -u origin main
On branch main
nothing to commit, working tree clean
No changes to commit
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 687 bytes | 85.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote: https://github.com/RMaheshBabu-RVCE24/program-4.git
To https://github.com/RMaheshBabu-RVCE24/Program-4.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
% rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg4$
```

This screenshot illustrates the sequence of commands executed while attempting to push the project code to GitHub, highlighting both the initial issues encountered and their successful resolution.

Linking the Local Repository to GitHub

Command: `git remote add origin ...`

Outcome: An error message stating “remote origin already exists” was displayed. This indicates that the local repository had already been connected to a GitHub remote, making it unnecessary to add the remote again.

Committing Changes

Command: `git commit -m "First commit"`

Outcome: The message “nothing to commit, working tree clean” was returned, confirming that all files had already been staged and committed prior to running this command.

Pushing to the Master Branch

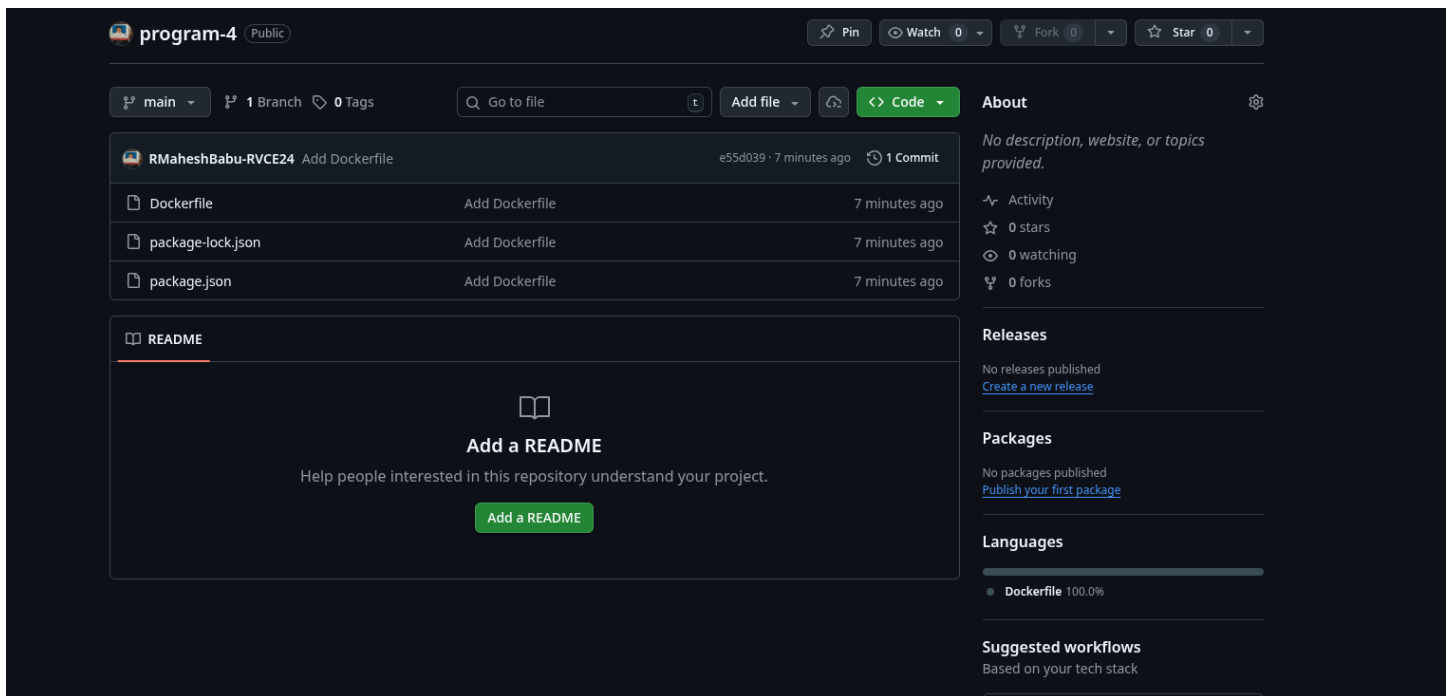
Command: `git push -u origin master`

Outcome: An error occurred stating “src refspec master does not match any”. This happened because GitHub now uses main as the default branch name instead of master, while the command attempted to push to a non-existent branch.

Pushing to the Correct Branch

Command: `git push -u origin main`

Outcome: The push operation completed successfully. The system prompted for authentication and then confirmed that the repository was synchronized, displaying the message “Everything up-to-date”.



This screenshot displays the remote GitHub repository named `rvceStudent/Program-4`, confirming that the project has been successfully pushed to GitHub.

The repository contains a standard Node.js project structure, including:

`package.json` and `package-lock.json` – Files used for dependency management and version control.

`Dockerfile` – The configuration file that defines the steps required to build the Docker container image.

The commit message **“Initial Program4 commit”** indicates that this is a newly initialized project and represents the first successful commit to the repository.